Data Scientist Review Guide

Thesis: *Lyric and Audio Emotional Alignment as a Predictor of Spotify Track Popularity: A Feature Engineering Approach*
Researchers: Joenell Conrad C. de Pedro and Ivan Paul V. Ruelan, BS Computer Science, University of San Carlos
Adviser: Christian V. Maderazo, M.Eng
Reviewer: Neriah "BJ" Ato, Data Scientist
Deadline: Monday, 19 October 2026

1. What we need from you

1. Read Sections 4.2.2 and 4.9 of the attached proposal (about 30 minutes).
2. Read Section 4 of this guide, which explains step by step how the model was built and trained.
3. Look over the modeling code and results in the repository (about 1 hour).
4. Optional: run the 2-minute reproduction check, or re-run the full pipeline (about 40 minutes of unattended compute).
5. Fill in the review form in Section 10.
6. Send the form back to us by **Monday, 19 October 2026**.

Total time: about 2 to 3 hours. You do not need a GPU, a database or audio software, and you do not need to write code.

The two questions you are answering (proposal Section 4.9.3(C), pp. 84 to 86):

1. Is the hyperparameter configuration chosen by our grid search a defensible choice for this dataset's size and features?
2. Are the reported R^2, RMSE and MAE figures correctly and reproducibly derived from that configuration?

This is not a second hyperparameter search. You do not need to find a better model. We need to know whether the figures we report can be trusted.

2. What happens with your review

- Your verdicts and comments are reported in Section 4.9.3(C) of the thesis as an independent expert assessment. They are reported in words, not converted to a score.
- Any reservation or disagreement is reported as you wrote it, together with our response or the fix we made.
- We may send you one short round of follow-up questions.
- If anything is unclear while you review, contact us instead of guessing (details at the end).

In return you receive:

- Named acknowledgement in Section 4.9.3(C) and in the Acknowledgements, with your consent (the form asks). You can choose to be credited by role only.
- A copy of the final results chapter and the defense outcome.
- `[Certificate of appreciation signed by the adviser: keep or delete]`
- `[Honorarium or token of appreciation: _____ : keep or delete]`

3. The study in brief

Research question. Does the agreement or mismatch between a song's lyrical mood and its musical mood help predict Spotify popularity, beyond the audio and lyric features themselves?

The engineered feature.

```
valence_normalized = 2 × valence − 1                      Spotify valence, rescaled from [0, 1] to [−1,
1]
alignment_gap      = lyric_sentiment − valence_normalized  range [−2, 2]
```

`lyric_sentiment` is the VADER compound score (−1 to +1). A positive gap means the lyrics are more positive than the music sounds.

Data. 259,458 English-language tracks released 2000 to 2023, from the Kaggle *490K Spotify Song Audio Embeddings & Metadata* dataset. The target is Spotify `popularity` (0 to 100), modeled as regression.

Models. Five feature configurations:

Model	Features
Baseline	none (predicts the training mean)
Audio-Only	the twelve Spotify audio descriptors (key one-hot encoded)
Lyric-Only	lyric sentiment
Control	audio + lyric sentiment
Alignment-Augmented	audio + lyric sentiment + `alignment_gap`

The four non-baseline models are trained with XGBoost (primary) and Random Forest (robustness check): nine models in total.

Decision rule. Control and Alignment-Augmented differ by one column. The alignment feature counts as a contribution only if the 95% paired bootstrap confidence interval on ΔR^2 (10,000 test-set resamples) excludes zero.

Result. The interval contains zero for both algorithms, so the thesis reports **no detectable contribution**. The proposal committed to this reporting rule in advance (Section 4.9.3(A)). Your job is to judge whether this null result is trustworthy: could the pipeline have hidden a real effect, or produced a false null?

Where the modeling sits in the project

#	Objective (proposal p. 12)	Where	In your review?
1	Clean the data: 491,632 tracks filtered to popularity > 0, years 2000 to 2023, duplicates removed, valid English lyrics only. 259,458 remain	`notebooks/1_validation`, `2_lyrics_filtering`; `docs/cleaning_process.md`	No (upstream)
2	Score lyric sentiment with VADER	`notebooks/3_sentiment_analysis`	No (upstream)
3	Compute `alignment_gap`, verified against hand-worked cases (max error 2.2×10^{-16})	`notebooks/3_sentiment_analysis`, `4_behavioral_checks`	No (upstream)
4	**Split, tune, train, evaluate, robustness checks**	**`notebooks/5_splits` to `7_model_ladder`, `9_robustness`, `modeling_config.py`**	**Yes**
5	SHAP attribution	`notebooks/8_shap`	Comments welcome
6	Demo web app	`api/`, `app/`	No

```
Kaggle dataset (491,632 tracks)
   |   notebooks 1 to 2: filtering
   v
259,458 tracks  (notebooks/data_filtered/songs_2000.csv to songs_2023.csv)
   |   notebook 3: lyric_sentiment, alignment_gap
   |   notebook 4: formula and behavioral checks
   v
+---------------------- YOUR REVIEW -----------------------+
|  notebook 5   artist-disjoint 70/15/15 split             |
|  notebook 6   grid searches                              |
|  notebook 7   nine models, bootstrap on delta R2         |
|  notebook 9   robustness checks                          |
+----------------------------------------------------------+
   |   notebook 8: SHAP
   |   scripts/export_popularity_model.py
   v
Demo app: POST /predict serves this same XGBoost model
```

The demo app is out of scope, with one exception. Its `/predict` endpoint serves the exact model you are reviewing. The export script only writes that model file if it matches notebook 7's test predictions to within 10^{-4}. The rest of the app estimates audio descriptors from raw audio for new songs; that is validated separately (on 66 real songs, estimation error moves the prediction by 2.65 points on average, against a model RMSE of about 15). Ignore `api/`, `app/` and `scripts/` except `export_popularity_model.py`.

A separate music teacher reviews the audio features (key, mode and valence on 15 sample tracks). You do not need to look at that material.

Context from notebook 4 (`behavioral_checks.json`):

Check	Result
Correlation of `alignment_gap` with lyric sentiment	0.857
Correlation with valence	−0.393
Correlation with popularity	0.019
Tracks where lyrics and music agree (|gap| ≤ 0.5)	29.3%

The gap is mostly a rescaled sentiment score, and it is an exact combination of two columns Control already has. Section 8, item 10 asks for your view on what that means for the test.

4. How the model was built and trained

This is the full procedure, in the order it runs. Every step is code in `notebooks/` and uses the shared definitions in `modeling_config.py`. The random seed is 42 throughout.

How each partition is used:

Partition	Rows	Used for	Never used for
Train	179,218	Fitting every model	Choosing settings or reporting results
Validation	37,784	Choosing hyperparameters; XGBoost early stopping	Reported results
Test	37,632	Final metrics, bootstrap, SHAP	Any fitting or choice of settings

Stage 1. Load the data (all notebooks)

`load_corpus()` joins the 24 yearly CSVs in `notebooks/data_filtered/` into one table of 259,458 tracks. The target is `popularity` (1 to 96 in this corpus, mean 24.5, standard deviation 15.9). There are no missing values in any modeling column.

Stage 2. Build the features (`build_feature_frame`)

- The eleven non-key audio descriptors are used as they are: tempo, loudness, mode, energy, danceability, speechiness, acousticness, instrumentalness, liveness, duration_ms, valence.
- `key` (0 to 11) is one-hot encoded into twelve columns, because pitch classes have no order. That gives 23 audio columns.
- `lyric_sentiment` (1 column) and `alignment_gap` (1 column) are added.
- No scaling or target transformation (tree models do not need it).
- `assert_clean()` stops the run if any forbidden column (year, genre, artist counts, `valence_norm`, IDs, lyrics text) or any missing value reaches a model.

Each model then takes only its own columns from the ladder (`columns_for`): Audio-Only 23, Lyric-Only 1, Control 24, Alignment-Augmented 25.

Stage 3. Split the data (notebook 5)

Goal: 70/15/15, no artist in more than one partition, genre and popularity balanced.

1. Build an artist graph: one node per artist, an edge between artists credited on the same track (48,206 artists, 69,150 edges).
2. Find collaboration communities with Louvain (`networkx`, resolution 3.0, seed 42): 19,500 communities, the largest holding 3.3% of tracks.
3. Assign whole communities to train, validation or test, largest first. Each goes to the partition it overfills least, scoring total size, genre mix and popularity-quintile mix together (popularity weighted 3×).
4. Drop any track whose credited artists ended up in different partitions: 4,824 tracks (1.86%).
5. Assert that no artist appears in two partitions, that each partition is within 2 points of its target share, that genre shares are within 3 points of the corpus, and that mean popularity differs by less than 1.5 points. The result is saved to `splits.csv` and `manifest.json`.

Stage 4. Choose the hyperparameters (notebook 6)

Tuning is done on the **Control** feature set only (24 columns): fit on train, score on validation, keep the setting with the lowest validation RMSE.

- **XGBoost:** 9 combinations of `max_depth` {4, 6, 8} and `learning_rate` {0.01, 0.05, 0.1}. Fixed: up to 1,000 trees, early stopping after 50 rounds without validation improvement, `subsample` 0.8, `colsample_bytree` 0.8, `tree_method` hist, squared-error loss. **Selected: depth 4, learning rate 0.05** (best round 549, validation RMSE 14.6408).
- **Random Forest:** 4 combinations of `n_estimators` {200, 500} and `min_samples_leaf` {1, 5}. Fixed: unlimited depth, `max_features` = ⌊p/3⌋. **Selected: 500 trees, minimum leaf size 5** (validation RMSE 14.7127).

The chosen settings are saved to `config_xgb.json` and `config_rf.json` and not changed again.

Stage 5. Train the nine models (notebook 7)

- **Baseline:** predicts the training-set mean popularity (24.53) for every test track. No fitting. Shared by both algorithms.
- **XGBoost** (4 models): each fitted on train with the frozen settings, early-stopping on validation. Each model finds its own stopping round, so round counts differ slightly between models.
- **Random Forest** (4 models): each fitted on train with the frozen settings. `max_features` follows ⌊p/3⌋, so it changes with the column count (1 for Lyric-Only).
- Each model predicts the 37,632 test tracks. Predictions go to `test_predictions.csv`, the models to `notebooks/artifacts/models/`.

Stage 6. Verify the models (notebook 7)

The notebook stops with an error if any of these fail:

- Control and Alignment-Augmented differ by exactly one column, `alignment_gap`. Control equals Audio-Only plus Lyric-Only.
- No forbidden column is in any model.
- Row counts per partition match `manifest.json`.
- All four models of each algorithm have identical saved settings (ignoring `max_features` and the feature count).

Stage 7. Score and compare (notebook 7)

- R^2, RMSE and MAE are computed on the test set by `regression_metrics` (written out by hand, not taken from a library).
- **Paired bootstrap:** draw 37,632 test rows with replacement, recompute R^2 for Control and Alignment-Augmented on the same rows, record the difference; repeat 10,000 times. The 2.5th and 97.5th percentiles form the 95% CI. If it excludes zero, the feature contributes; if not, the result is "no detectable contribution".
- The same comparison is also broken down by genre and popularity band (`metrics_by_genre.csv`, `metrics_by_band.csv`).

Stage 8. Robustness checks (notebook 9, XGBoost, Control vs Alignment-Augmented)

- **Seeds:** retrain the pair with seeds 42, 7, 1337, 2024 and 99.
- **5-fold CV:** re-partition all modeled rows into 5 artist-disjoint folds (same community method). For each fold, train on the other four and test on it. Early stopping uses a random 15% slice of the training folds, used only to pick the round count.
- **Hyperparameters:** retrain the pair under three other settings (depth 6 with rate 0.01, depth 8 with rate 0.1, and no subsampling).
- **Cross-algorithm:** compare the sign of ΔR^2 for XGBoost and Random Forest.

Stage 9. SHAP attribution (notebook 8)

`shap.TreeExplainer` computes exact SHAP values for the XGBoost Alignment-Augmented model on all 37,632 test tracks. The notebook checks that the SHAP values plus the base value reproduce each prediction. Mean |SHAP| is summed per feature group (audio, lyric sentiment, alignment), and the twelve key columns are merged back into one `key` feature for the ranking.

Stage 10. Export the model for the demo app (`scripts/export_popularity_model.py`)

The script retrains XGBoost Alignment-Augmented exactly as in Stage 5 and checks its test predictions against notebook 7's. If they match within 10^{-4}, it saves the model in XGBoost's JSON format to `models/popularity/`, which the demo app's `POST /predict` loads. This is the quick check in Step 3 below.

5. Setup

You need Git, Python 3.13, `uv`, 16 GB RAM and about 15 GB of free disk. Any OS works.

```
git clone https://github.com/grenappl/thesis-project.git
cd thesis-project
uv sync
```

The repository is public. The dataset (about 400 MB) is included.

`uv sync` takes 15 to 20 minutes because the environment includes PyTorch and Transformers for the demo app. Use it anyway: `uv.lock` pins every version, and the artist split depends on the exact `networkx` version.

Ignore Section 4 of `MODELING_REPORT.md`. It is a pre-run planning note describing an older split design. The current split is the one in notebook 5 and this guide.

6. Review steps

Step 1. Read the proposal (30 minutes)

Section	Pages	Content
4.2.1 (end)	59 to 61	Alignment formula, artist-aware split
4.2.2	61 to 66	Algorithms, hyperparameter grids (p. 63), model ladder (Table 5), metrics
4.9.1 to 4.9.2	79 to 81	Verification and robustness testing
4.9.3(A)	82 to 84	Bootstrap decision rule; **Table 11** (p. 83), the expected ranges set before training
4.9.3(C)	84 to 86	Your role

Correction: p. 61 calls the audio descriptors "self-computed". They are not. The models use the descriptors Spotify supplies in the dataset (see Section 4.3, p. 66).

Step 2. Read the code (20 minutes)

All in `notebooks/`. Each notebook reads what the previous one wrote to `notebooks/artifacts/`.

File	Content
`modeling_config.py`	Start here. Feature lists, the model ladder, forbidden columns, metric functions, the split function
`5_splits.ipynb`	Artist-disjoint 70/15/15 split, with assertions
`6_tuning.ipynb`	Both grid searches
`7_model_ladder.ipynb`	The nine models, verification cell, bootstrap. Headline figures come from here
`8_shap.ipynb`	SHAP on the XGBoost Alignment-Augmented model
`9_robustness.ipynb`	Seed, CV, hyperparameter and cross-algorithm checks

Check these three things in the code:

- Control's columns exclude `alignment_gap` (asserted in notebook 7's verification cell).
- R^2, RMSE and MAE are computed by hand in `regression_metrics`, not by a library call.
- `paired_bootstrap_delta_r2` (notebook 7) scores both models on the same resampled rows.

Step 3. Reproduce the results (optional)

Quick check (2 minutes). Retrains the main model and compares it with notebook 7's saved test predictions:

```
uv run python scripts/export_popularity_model.py
```

Expected output:

```
trained on 179,218 rows, 25 features, stopped at round 517
test R2 +0.06783  RMSE 15.0927  MAE 12.1144
max |retrained - notebook 7| over 37,632 test tracks: ~1e-06
wrote models/popularity/xgboost_alignment_augmented.json
wrote models/popularity/model_metadata.json
```

A difference near 10^{-6} is float32 rounding. Above 10^{-4}, the script prints `FAIL` and writes nothing. Our last run (16 October 2026) gave 1.9×10^{-6}. Stopping at round 517 rather than 549 is expected: 549 is where Control stopped during tuning, and each model early-stops on its own.

Full check (about 40 minutes). Run notebooks 5 to 9 in order, using Run All in Jupyter or VS Code, or:

```
for nb in 5_splits 6_tuning 7_model_ladder 8_shap 9_robustness; do
  uv run --with nbconvert --with ipykernel jupyter nbconvert \
    --to notebook --execute --inplace "notebooks/$nb.ipynb"
done
```

(In PowerShell, run the `uv run` line once per notebook.) Compare the new values with Section 7. R^2 should match to 4 decimal places, RMSE and MAE to 2.

Step 4. Judge the configuration

Evidence: `tuning_results.csv`, `config_xgb.json`, `config_rf.json`, and Section 8.

- XGBoost grid: `max_depth` {4, 6, 8} × `learning_rate` {0.01, 0.05, 0.1}, up to 1,000 rounds with 50-round early stopping, `subsample` and `colsample_bytree` fixed at 0.8. Reasonable for 179,000 rows and 25 features?
- Random Forest grid: `n_estimators` {200, 500} × `min_samples_leaf` {1, 5}, unlimited depth, `max_features` = ⌊p/3⌋. Reasonable?
- Is one fixed 15% validation partition, instead of k-fold CV, acceptable at this scale?
- Is tuning on Control and reusing that configuration for every model fair?

Step 5. Judge the derivation

Evidence: your Step 3 run (or Section 7), notebook 7's verification cell, notebook 5's assertions.

- Do the figures reproduce?
- Is the split free of artist leakage?
- Is the paired bootstrap on ΔR^2 a valid decision rule here?
- Do the robustness results support "no detectable contribution"?

Step 6. Fill in the form

Use Section 10. Short comments are fine. Explain any reservation or disagreement, since that is what we report.

7. Reference figures

All from `notebooks/artifacts/`.

Split (`manifest.json`)

Partition	Tracks	Share	Mean popularity
Train	179,218	70.4%	24.53
Validation	37,784	14.8%	23.78

Partition	Tracks	Share	Mean popularity
Test	37,632	14.8%	23.97
Excluded (collaborations spanning partitions)	4,824	1.86% of corpus	n/a

No artist, including featured artists, appears in more than one partition. Genre shares are within 1.95 percentage points of the full corpus.

Tuning (`tuning_results.csv`)

Algorithm	Selected	Validation RMSE
XGBoost	`max_depth` 4, `learning_rate` 0.05 (best round 549)	14.6408
Random Forest	`n_estimators` 500, `min_samples_leaf` 5	14.7127

All nine XGBoost settings scored between 14.641 and 14.690, so the grid is nearly flat.

Test-set results (`ladder_metrics.csv`)

Algorithm	Model	Features	R^2	RMSE	MAE
shared	Baseline	0	−0.0013	15.642	12.766
XGBoost	Audio-Only	23	0.0671	15.098	12.121
XGBoost	Lyric-Only	1	0.0067	15.580	12.688
XGBoost	Control	24	0.0683	15.089	12.109
XGBoost	Alignment-Augmented	25	0.0678	15.093	12.114
Random Forest	Audio-Only	23	0.0619	15.141	12.169
Random Forest	Lyric-Only	1	−0.0370	15.919	12.916
Random Forest	Control	24	0.0623	15.137	12.166
Random Forest	Alignment-Augmented	25	0.0626	15.135	12.160

23 audio columns = eleven descriptors plus twelve one-hot key columns.

Central comparison (`central_comparison.json`)

Algorithm	ΔR^2 (Augmented minus Control)	95% bootstrap CI	Excludes zero?
XGBoost	−0.00042	[−0.00115, +0.00032]	No
Random Forest	+0.00027	[−0.00072, +0.00123]	No

Against proposal Table 11 (p. 83)

Quantity	Expected	Observed (XGBoost)	Reading
Baseline R^2	about 0, possibly slightly negative	−0.0013	As expected
Control R^2	0.05 to 0.20	0.0683	In range. Above the 0.05 "broken pipeline" threshold, well below the 0.20 "possible leakage" threshold
Control RMSE	15 to 20	15.09	In range

Quantity	Expected	Observed (XGBoost)	Reading
ΔR^2	0.001 to 0.01	−0.0004 (−0.61% of Control R^2)	Below range
95% CI on ΔR^2	decides the outcome	[−0.0011, +0.0003]	Contains zero: no detectable contribution

Robustness (`robustness_summary.json`)

Check	Result
5 seeds	ΔR^2: −0.00042, +0.00058, −0.00106, +0.00096, −0.00084. Mean (−0.00016) is smaller than the seed-to-seed R^2 standard deviation (0.00051)
5-fold artist-disjoint CV	ΔR^2: +0.00151, −0.00086, −0.00049, −0.00039, +0.00063. Control R^2 ranges 0.023 to 0.165 across folds
4 hyperparameter settings	ΔR^2: −0.00042 (selected), +0.00031, −0.00098, −0.00018. No consistent sign
XGBoost vs Random Forest	Opposite signs

SHAP (`shap_group_contributions.csv`, `shap_feature_ranks.csv`)

Audio descriptors: 91.5% of mean |SHAP|. Lyric sentiment: 6.7%. Alignment score: 1.7%, ranking 13th of 14 features.

8. Decisions to check

These are judgement calls we made. Some are not yet stated in the proposal.

1. **Tuning used the Control configuration.** The proposal does not specify which. We chose Control so tuning could not favour the treatment model. The result was reused for all nine models.
2. **Each model early-stops on its own.** The configuration is shared, but XGBoost's round count differs per model. We treat it as an outcome of early stopping, not a tuned value.
3. **Random Forest `max_features` = ⌊p/3⌋ varies with feature count.** Lyric-Only has p = 1.
4. **The split uses community detection.** Collaborations link artists into one group covering 51% of tracks, so splitting by connected groups would give an unrepresentative split. We detect collaboration communities (Louvain), assign whole communities to partitions while balancing size, genre and popularity, and drop the 1.86% of tracks whose artists end up in different partitions. Dropped tracks are all collaborations, more often Electronic and Hip-Hop, and more popular on average (33.6 vs 24.3).
5. **No feature scaling or target transformation.** Popularity is modeled on its raw 0 to 100 scale.
6. **Excluded features:** release year, genre (stratification only), and artist follower and popularity counts (they leak artist identity). Asserted in code.
7. **The bootstrap uses one test partition**, so it cannot capture split-to-split variation. The 5-fold CV covers that, and shows absolute R^2 depends heavily on the partition (0.023 to 0.165).
8. **Two grid points hit the round cap.** Both `learning_rate` 0.01 settings stopped at round 998 or 999 of 1,000, so early stopping never triggered. We did not raise the cap, because the whole grid spans only 0.05 RMSE and the robustness check at `learning_rate` 0.01 does not change the conclusion. Tell us if you think the cap should have been higher.
9. **Absolute R^2 is low (about 0.07).** Popularity depends heavily on artist reach, marketing and playlisting, none of which the models receive. The thesis tests the difference between two models, not the absolute level.

10. `alignment_gap` **is an exact combination of two Control inputs.** The proposal argues (p. 61) that trees use a difference more easily when it is supplied directly. Given the null result and the 0.857 correlation with sentiment, is the ablation still a fair test of the feature, or mainly a test of whether the trees already recover the difference? Please comment.
11. **The validation partition does two jobs.** It picks the hyperparameters and also sets each XGBoost model's early-stopping round. Validation RMSE is therefore slightly optimistic. The test set is used for neither, so the reported test figures are not affected.

9. Errors we already fixed

1. **Per-popularity-band R^2.** Inside a 20-point band, popularity barely varies, so R^2 was meaningless (−320 for the top band). That table now reports RMSE and MAE only.
2. **A duplicate hyperparameter setting.** One "alternative" in the robustness check was identical to the selected configuration. We replaced it, and the hyperparameter check no longer shows a consistent direction. The notebook now asserts that no alternative equals the selected one.

Neither fix required retraining the nine models or changed the headline result.

10. Review form

For each statement, mark **Agree**, **Agree with reservations**, **Disagree** or **Not assessed**. Add a comment for anything other than Agree.

A. Hyperparameter configuration

#	Statement	Verdict	Comments
A1	The XGBoost search space suits this dataset's size and features.		
A2	The Random Forest search space suits this dataset's size and features.		
A3	Selecting on a single fixed 15% validation partition is defensible at this scale.		
A4	Tuning on Control and reusing that configuration for every model keeps the comparison fair.		
A5	The selected configurations are defensible overall.		

B. Reported figures

#	Statement	Verdict	Comments
B1	The reported R^2, RMSE and MAE reproduce from the configuration. *(State whether you ran the quick check, the full check, or reviewed the figures only.)*		
B2	The metrics are computed correctly.		
B3	The split is free of artist-identity leakage.		

#	Statement	Verdict	Comments
B4	Excluding the 1.86% of cross-partition collaborations is acceptable and adequately reported.		
B5	The paired bootstrap CI on ΔR^2 is a valid decision rule for this comparison.		
B6	The robustness checks support "no detectable contribution".		

C. Overall

#	Statement	Verdict	Comments
C1	The reported model accuracy can be trusted, given how it was produced.		

Other concerns or suggested changes:

_____

_____

_____

Consent to be named. May we name you and state your professional background in the thesis? Tick one.

☐ Yes. State my background as: _____
☐ Credit me by role only ("an independent data scientist")
☐ No

Reviewer: Neriah "BJ" Ato
Signature: _____
Date: _____

11. File locations

What	Where
Modeling notebooks	`notebooks/5_splits.ipynb` to `notebooks/9_robustness.ipynb`
Shared definitions	`notebooks/modeling_config.py`
All results	`notebooks/artifacts/`
Split record	`manifest.json`
Grid results and selected configs	`tuning_results.csv`, `config_xgb.json`, `config_rf.json`
Model metrics	`ladder_metrics.csv`
Bootstrap comparison	`central_comparison.json`
Robustness	`robustness_*.csv`, `robustness_summary.json`
SHAP	`shap_*.csv`, `fig_shap_*.png`
Test predictions (all nine models)	`test_predictions.csv`
Reproduction script	`scripts/export_popularity_model.py`

What	Where
Upstream cleaning and sentiment (context)	`notebooks/1_validation.ipynb` to `4_behavioral_checks.ipynb`, `docs/cleaning_process.md`
Demo app (out of scope)	`api/`, `app/`; audio-estimation validation in `docs/feature_extraction_calibration.md`

Files under `notebooks/artifacts/` are listed by name only.

Contact: Joenell Conrad C. de Pedro or Ivan Paul V. Ruelan, `[email or phone]`. We can also set up a short call.